IOT PRACTICUM JOURNAL

MODULE 1

BLUETOOTH LOW ENERGY (BLE) CONNECTIVITY

Group : .....

Day/Shift : .....

Instructor Assistant Code : .....

Name : .....

NIM : .....

IOT DEVICE PROGRAMMING PRACTICUM

SCHOOL OF ELECTRICAL ENGINEERING

TELKOM UNIVERSITY

2026

A. Circuit and System

1. Prepare the following components:

   - ESP32 DevKit V1 (DOIT) or ESP32-S3 (2x)
   - 10 kΩ Potentiometer (1x)
   - SG90 Servo (1x)
   - Breadboard (2x)
   - Wires/Jumpers (as needed)

2. Build the following circuit using the tools and components you have prepared! (POT: Potentiometer, SRV: Servo)

Module 1 circuit schematic (to follow)

Central / Client		Peripheral / Server	
Component	**Pin**	**Component**	**Pin**
POT (left leg)	GND	SRV (brown)	GND
POT (wiper)	GPIO34	SRV (red)	VIN (5V)
POT (right leg)	3V3	SRV (orange)	GPIO16

Note: GPIO34 is an input-only pin and belongs to ADC1, the only ADC that can still be read while the BLE radio is on. An SG90 servo draws an inrush current the 3V3 regulator cannot supply, so its red wire must be taken from the VIN (5 V) pin with a common ground. The ESP32-S3 has no ADC on GPIO34, so on that board the potentiometer moves to GPIO4 by enabling `#define BOARD_ESP32S3` in the Central program.

UUID assignment per group. Every group uses its own pair of UUIDs. In **both** programs, remove the `//` in front of your own group's pair only and leave the other pairs commented out. If two groups use the same pair, one group's Central can connect to another group's Peripheral.

Group	`SERVICE_UUID` *(last 4 characters)*	`CHARACTERISTIC_UUID` *(last 4 characters)*
1	`ce01`	`6501`
2	`ce02`	`6502`
3	`ce03`	`6503`
4	`ce04`	`6504`
5	`ce05`	`6505`
6	`ce06`	`6506`

Group	`SERVICE_UUID` *(last 4 characters)*	`CHARACTERISTIC_UUID` *(last 4 characters)*
7	`ce07`	`6507`
8	`ce08`	`6508`

3. Before running the program, prepare your working environment in the following order. Tick every step you have completed!

   - ☐ Installing the `ESP32Servo` library through the Arduino IDE Library Manager
   - ☐ Selecting the ESP32 Dev Module board in the Arduino IDE for both boards
   - ☐ Activating your group's UUID pair in **both** programs (see the table above)
   - ☐ Removing the `//` in front of `#define BOARD_ESP32S3` when using an ESP32-S3 board
   - ☐ Connecting the servo's red wire to VIN, not to 3V3
   - ☐ Syncing **both** boards' clocks from the same PC: `.\scripts\send_time.ps1 -Port COM_A,COM_B`

4. Complete the program below by filling in the blank sections. Afterwards (or while completing it), retype the blank sections you have filled in using the VS Code/Arduino IDE application prepared by your respective lab assistants!

   Blank sections are marked with an underscore run `________` and numbered `(1)` through `(6)` in the comment to their right. Replace each underscore run with the correct code following the hint in its comment. Remember that the service UUID and the characteristic UUID on the Central side must match the values on the Peripheral side exactly.

Central/Client Source Code — Bluetooth Low Energy (ESP32)

```
 */

#include <BLEDevice.h>
#include <BLEUtils.h>
#include <________>  // (1) Header file for the BLE device scanner
#include <BLEAdvertisedDevice.h>
#include <time.h>

// ---- GROUP UUIDS ------------------------------------------------------------
…
}

// Turns the voltage on the potentiometer into a servo angle.
int readAngle() {
  int raw = ________(POT_PIN);  // (2) Function that reads a pin's analog value

  return map(raw, 0, ADC_MAX, SERVO_MIN_ANGLE, ________);  // (3) Highest angle limit
}

// Called by the BLE library every time the Peripheral sends a notification.
// The echo arrives here, so the round-trip delay is measured here too.
…
    return false;
  }

  // A service is the container, a characteristic is the data inside it.
  BLERemoteService *pService = pClient->getService(________);  // (4) Service UUID

  if (pService == nullptr) {
    Serial.println("Service not found!");
    pClient->disconnect();
…
  }
```

```
234
235     // Without registering here, the echo from the Peripheral never arrives and
236     // the Delay column in the journal cannot be filled in.
237     pRemoteCharacteristic->registerForNotify(________);  // (5) Notification function name
238
239     connected = true;
240     Serial.println("Ready! Turn the potentiometer to move the servo.");
241     return true;
    …
306       lastSendTime = sendTime;
307       lastSentAngle = angle;
308       waitingForEcho = true;
309
310       String payload = String(________);  // (6) Angle variable to be sent
311
312       Serial.printf("[%s] Potentiometer moved, sending angle: %s degrees\n",
313                     formatTimestamp(sendTime).c_str(), payload.c_str());
```

Peripheral/Server Source Code — Bluetooth Low Energy (ESP32)

```
10   * one are the hints.
11   */
12
13  #include <BLEDevice.h>
14  #include <________>  // (1) Header file that creates a BLE server/peripheral
15  #include <BLEUtils.h>
16  #include <BLE2902.h>
17  #include <ESP32Servo.h>
18  #include <time.h>
    …
155       // Echo the data as early as possible, before driving the servo and printing
156       // to Serial, so the round-trip measurement on the Central is not weighed down.
157       pCharacteristic->setValue(data.c_str());
158       pCharacteristic->________();  // (2) Method that sends a value to the Central
159
160       servo.write(________);  // (3) Angle variable parsed from the data
161
162       Serial.printf("[%s] Angle from Central: %d degrees | Servo moved\n",
163                     formatTimestamp(receiveTime).c_str(), angle);
164     }
    …
179     // PROPERTY_WRITE_NR -> the Central sends data without confirmation
180     pCharacteristic = pService->createCharacteristic(
181       BLEUUID(CHARACTERISTIC_UUID),
182       BLECharacteristic::PROPERTY_WRITE |
183       BLECharacteristic::________  // (4) Property that sends data automatically without ACK
184     );
185
186     // Descriptor 0x2902 is what lets the Central switch notifications on.
187     pCharacteristic->addDescriptor(new BLE2902());
188     pCharacteristic->setCallbacks(new ________());  // (5) Write callback class name
189     pCharacteristic->setValue("0");
190
191     pService->start();
    …
202   void setup() {
203     Serial.begin(115200);
204     syncTimeFromSerial();
205
206     servo.attach(________, SERVO_MIN_US, SERVO_MAX_US);  // (6) Servo pin constant
207     servo.write(90);
208
209     BLE_init();
210   }
```

5. Record the configuration you used in the following table! These two UUIDs are what must match exactly between the Central and Peripheral sides for a connection to form, because BLE filters its counterpart by the service UUID, not by the device name.

Parameter	Value used
Central device name (`centralName`)	
Peripheral device name (`peripheralName`)	
Group number and its UUID pair	
`SERVICE_UUID` *(first 4 characters only)*	
`CHARACTERISTIC_UUID` *(first 4 characters only)*	
Potentiometer pin on the Central (`POT_PIN`)	
Servo pin on the Peripheral (`SERVO_PIN`)	
Angle change threshold (`DEADBAND_DEG`)	

B. Analysis

1. Draw the communication flow implemented in the system you have worked on!

2. Next, a response-time experiment will be carried out for data sent from the **Central/Client** to the **Peripheral/Server**. First open **TWO (2)** Serial Monitors to observe the output of the **Central/Client** and the **Peripheral/Server**. Every time the potentiometer is turned far enough, the Central Serial Monitor prints the **send time** along with the angle sent, then prints the **round-trip** delay as soon as the echo from the Peripheral arrives; meanwhile the Peripheral Serial Monitor prints the **receive time** along with the servo angle applied. Record:
   - **FIVE (5)** pairs of readings: the send time and angle from the Central Serial Monitor, together with the receive time and servo angle from the Peripheral Serial Monitor

   Both times are recorded in **HH:MM:SS:MS** format. Compute the **one-way delay** by subtracting the send time from the receive time, then compare it against the round-trip delay the Central prints for itself.

   *Note: neither board has Internet access, so their clocks are set from the PC clock over Serial. As long as **both boards are synced from the same PC** (run `send_time.ps1` with both COM ports at once), the two clocks agree to within milliseconds and times from two different Serial Monitors **may** be subtracted. Serial synchronisation is accurate to a few milliseconds. If the boards were synced from different PCs, or one of them failed to receive the PC time (it prints a "time since boot" message), the one-way delay column is not valid and only the round-trip column can be used.*

Trial no. …	Servo angle **[degrees]**	Send time *(Central)* **[HH:MM:SS:MS]**	Receive time *(Peripheral)* **[HH:MM:SS:MS]**	One-way delay **[ms]**	Round-trip delay **[ms]**
1.					
2.					
3.					
4.					
5.					
Average delay (ms)					

3. Write your **analysis** of the system you experimented with earlier in the fill-in section below!

(Also give the **relationship/correlation** by applying the **communication methodology** you are studying in this module; it does not have to be complete!)

C. Suggestions and Feedback

1. Assistant

2. Laboratory